

Q.No:7

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A .

Note: Subarray means contiguous elements with at least one element in it.

Constraints

$$1 \leq N \leq 2000$$

$$0 \leq |A_i| \leq 10^9$$

Input Format

The first line of the input contains a single integer N , the total number of elements in array A .

The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
5 -2 7 -3
```

EXPECTED OUTPUT

```
17
```

Test Case 2

INPUT (STDIN)

```
15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
```

EXPECTED OUTPUT

```
2021
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int NA[N];
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b)
{
    long long x = *(long long *)a;
    long long y = *(long long *)b;

    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main()
```

```
{
int N;
scanf("%d", &N);

long long A[N];

for (int i = 0; i < N; i++)
{
scanf("%lld", &A[i]);
}

long long totalSubarrays = (long long)N * (N + 1) / 2;

long long *values = malloc(totalSubarrays * sizeof(long long));
long long count = 0;
for (int i = 0; i < N; i++)
{
long long currentSum = 0;
long long maxSum = 0;

for (int j = i; j < N; j++)
{
if (j == i)
{
currentSum = A[j];
maxSum = A[j];
}
else
{
if (currentSum + A[j] > A[j])
currentSum = currentSum + A[j];
else
currentSum = A[j];

if (currentSum > maxSum)
maxSum = currentSum;
}

values[count++] = maxSum;
}
}
qsort(values, count, sizeof(long long), cmp);
long long answer = values[0];

for (long long i = 1; i < count; i++)
{
if (values[i] != values[i - 1])
{
answer += values[i];
}
}

printf("Output:%lld\n", answer);
```

```
free(values);  
  
return 0;  
}
```

Output:

Test_case:1

```
4  
5 -2 7 -3  
Output:17
```

Test_case:2

```
15  
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9  
Output:2021
```